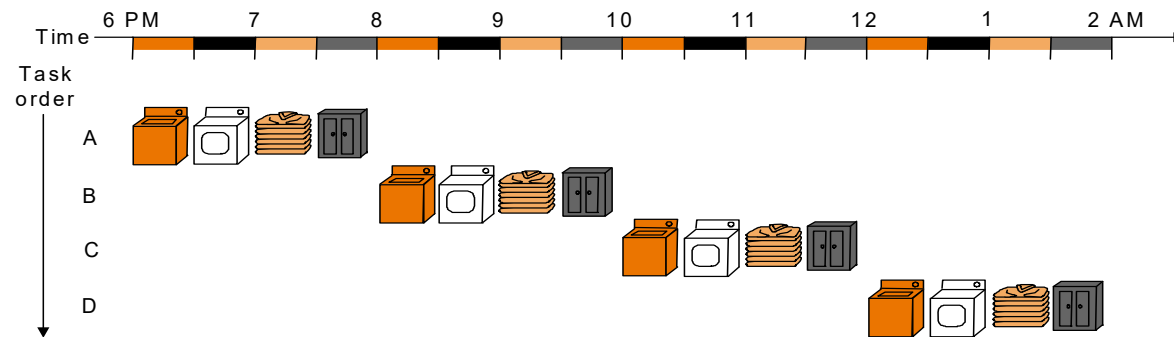
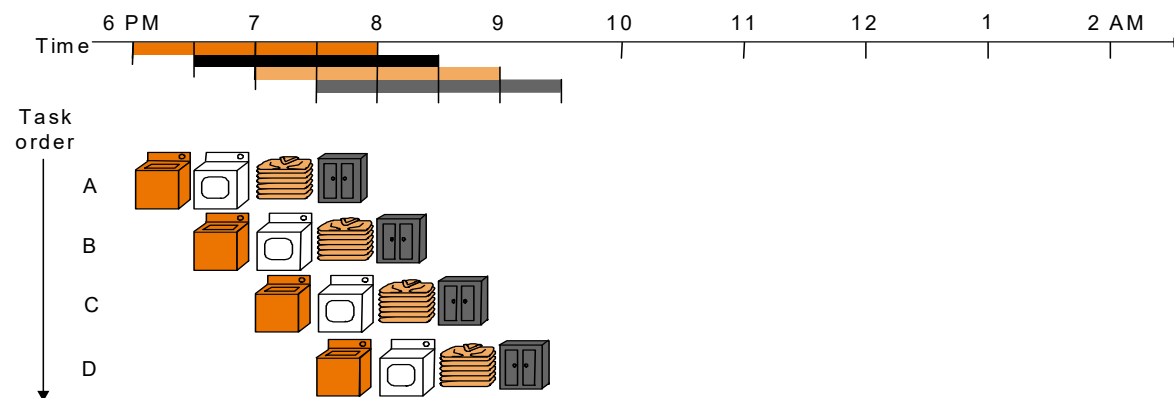


Pipelining

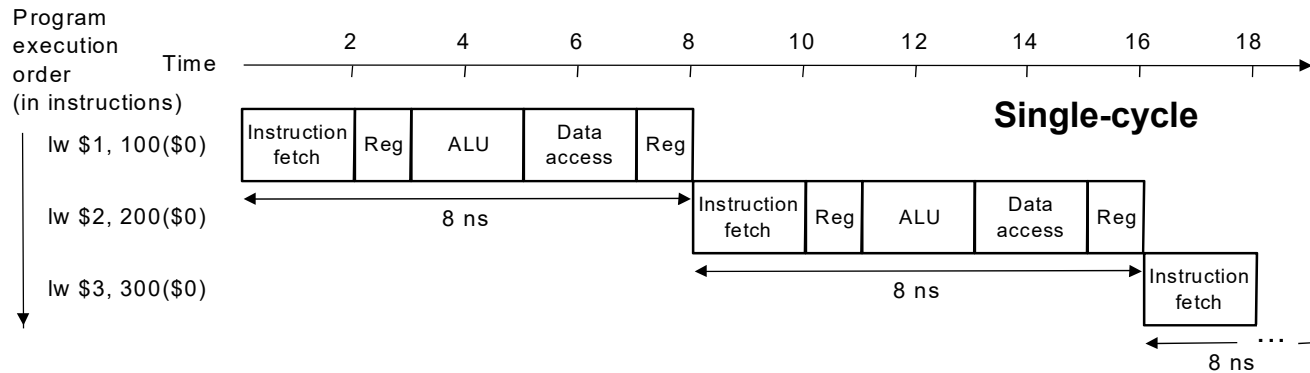
- Start work ASAP!! Do not waste time!



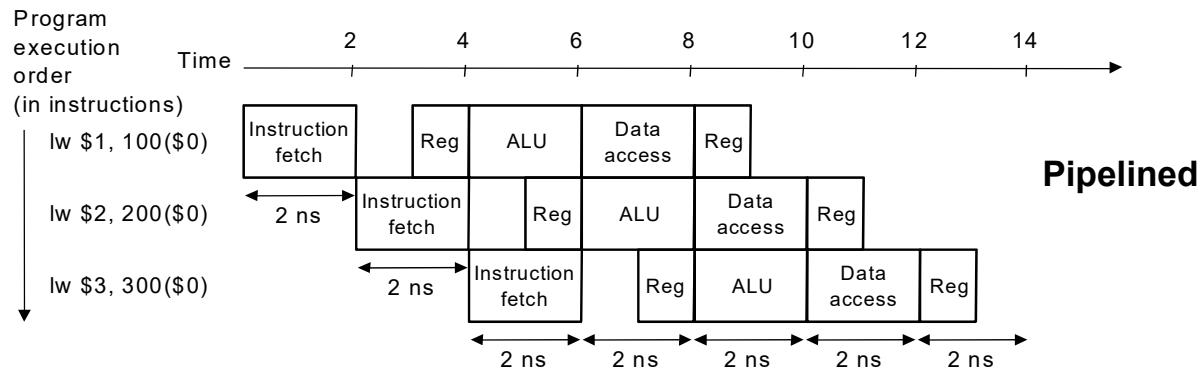
Assume 30 min. each task (wash, dry, fold, store) and that separate tasks use separate hardware and so tasks can be overlapped



Pipelined vs. Single-Cycle Instruction Execution: the Plan



Assume 2 ns for memory access, ALU operation; 1 ns for register access: therefore, single cycle clock 8 ns; pipelined clock cycle 2 ns.



Pipelining: Keep in Mind

- Pipelining *does not reduce latency* of a single task, which means pipelining does NOT reduce the time for a single task. It remains same but total time consumption lessens.
- Pipeline rate *limited by longest stage*
 - *Potential* speedup = Number of Stages
 - *Unbalanced lengths* of pipe stages reduces speedup (Thus, ideal case is all steps takes same time)
- Time to *fill* pipeline and time to *drain* it – when there is *slack* in the pipeline – reduces speedup:
 - In the beginning, you're still filling the pipeline – no output yet.
 - At the end, you're draining it – not all workers are busy.
 - This “slack time” reduces speedup, especially for small jobs.

Example Problem: Answer

- *Problem: for the laundry, fill in the following table when*
 - 1. the stage lengths are 30, 30, 30 30 min., resp.*
 - 2. the stage lengths are 20, 20, 60, 20 min., resp.*

Person	Unpipelined finish time	Pipeline 1 finish time	Ratio unpipelined to pipeline 1	Pipeline 2 finish time	Ratio unpipelelined to pipeline 2
1	120	120	1	120	1
2	240	150	1.6	180	1.33
3	360	180	2	240	1.5
4	480	210	2.29	300	1.60
n	120 x n			60 x (n+1)	

Pipelining MIPS

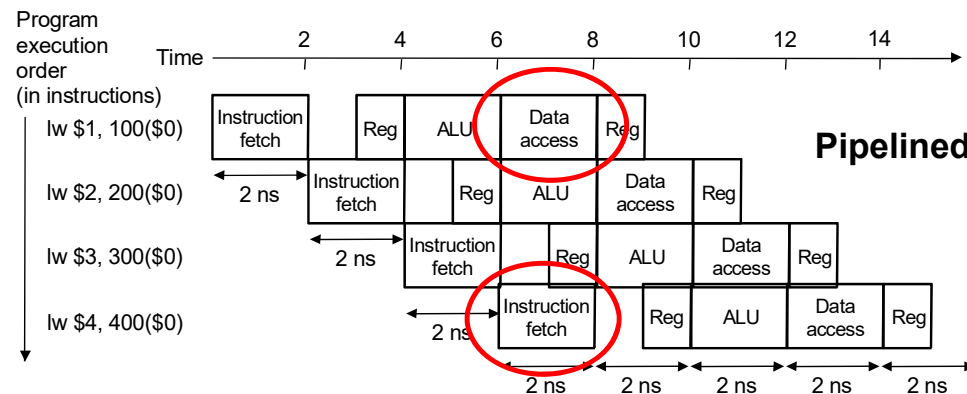
- What makes it easy with MIPS?
 - all *instructions are same length*
 - so fetch and decode stages are similar for all instructions
 - just a *few instruction formats*
 - simplifies instruction decode and makes it possible in one stage
 - *memory operands appear only in load/stores*
 - so memory access can be deferred to exactly one later stage
 - *operands are aligned in memory*
 - one data transfer instruction requires one memory access stage

Pipelining MIPS

- What makes it hard?
 - *structural hazards*: different instructions, at different stages, in the pipeline want to use the same hardware resource
 - *control hazards*: succeeding instruction, to put into pipeline, depends on the outcome of a previous branch instruction, already in pipeline
 - *data hazards*: an instruction in the pipeline requires data to be computed by a previous instruction still in the pipeline
- Before actually building the pipelined datapath and control we first briefly examine these potential hazards individually...

Structural Hazards

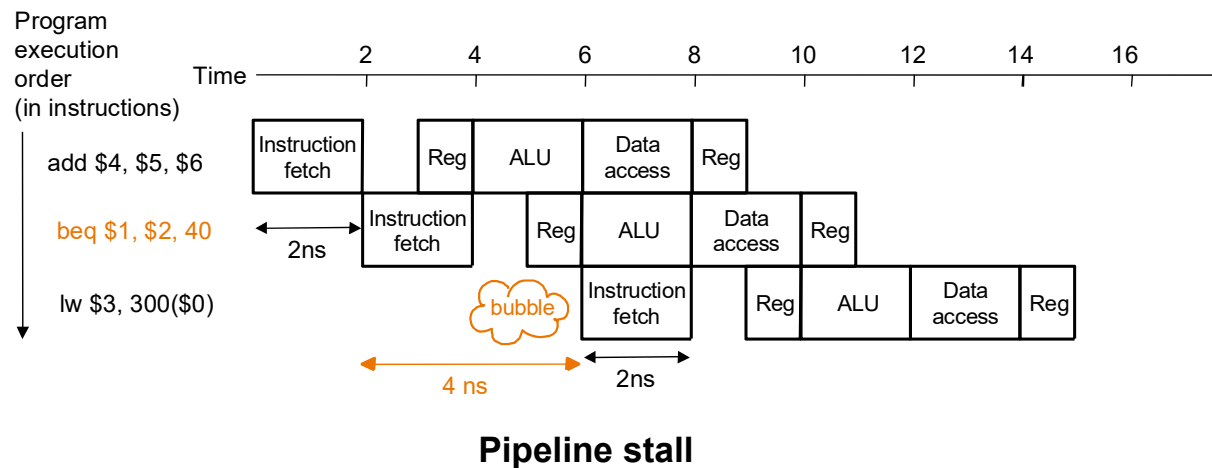
- *Structural hazard*: inadequate hardware to simultaneously support all instructions in the pipeline in the same clock cycle
- E.g., suppose *single – not separate* – instruction and data memory in pipeline below with *one read port*
 - then a structural hazard between first and fourth `lw` instructions



- *MIPS was designed to be pipelined*: structural hazards are easy to avoid!

Control Hazards

- *Control hazard*: need to make a decision based on the result of a previous instruction still executing in pipeline
- Solution 1 *Stall* the pipeline



Data Hazard

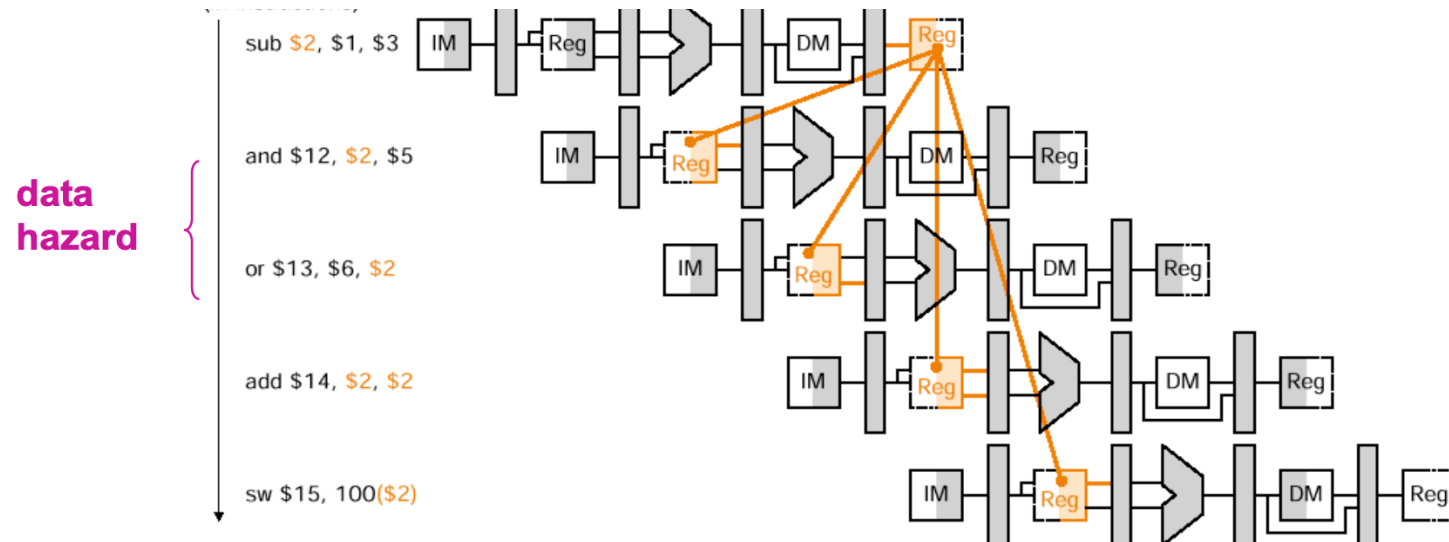
- An instruction depends on the results of a previous instruction still in pipeline.

ex) add **\$s0**, \$t0, \$t1

sub \$t2, **\$s0**, \$t3

- For example, if we have these instructions, \$s0 register cannot be used in sub instruction.
- Problem with starting next instruction before first is finished.

Data Hazard

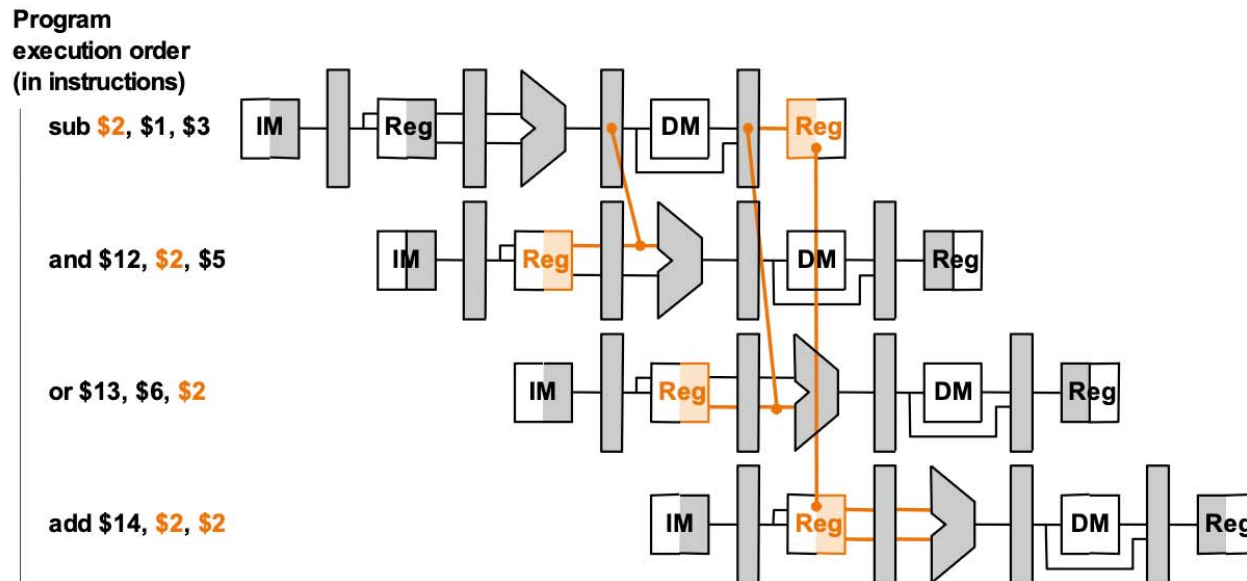


Solutions for Data Hazards

Stalling: We can stall other instructions until previous instruction executed.

Forwarding: We can create the path to send the result of instruction or calculation.

Data Hazard: Forwarding



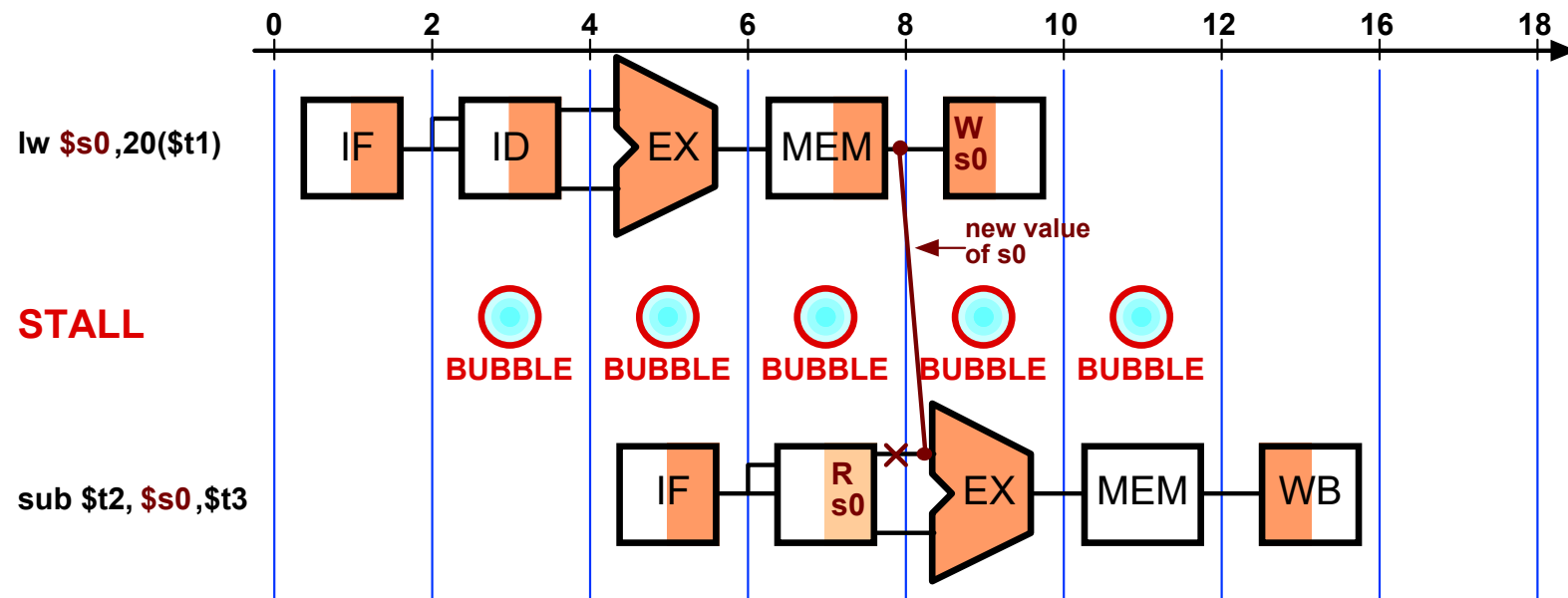
Pipeline stages of MIPS instruction

The 5 steps in instruction execution

1. Instruction **Fetch** & PC Increment (IF)
2. Instruction **Decode** and Register Read (ID)
3. **Execution** or calculate address (EX)
4. **Memory access** (MEM)
5. **Write** result into register (WB)

Data Hazards - Stalling

- STALL still required for load - data avail. after



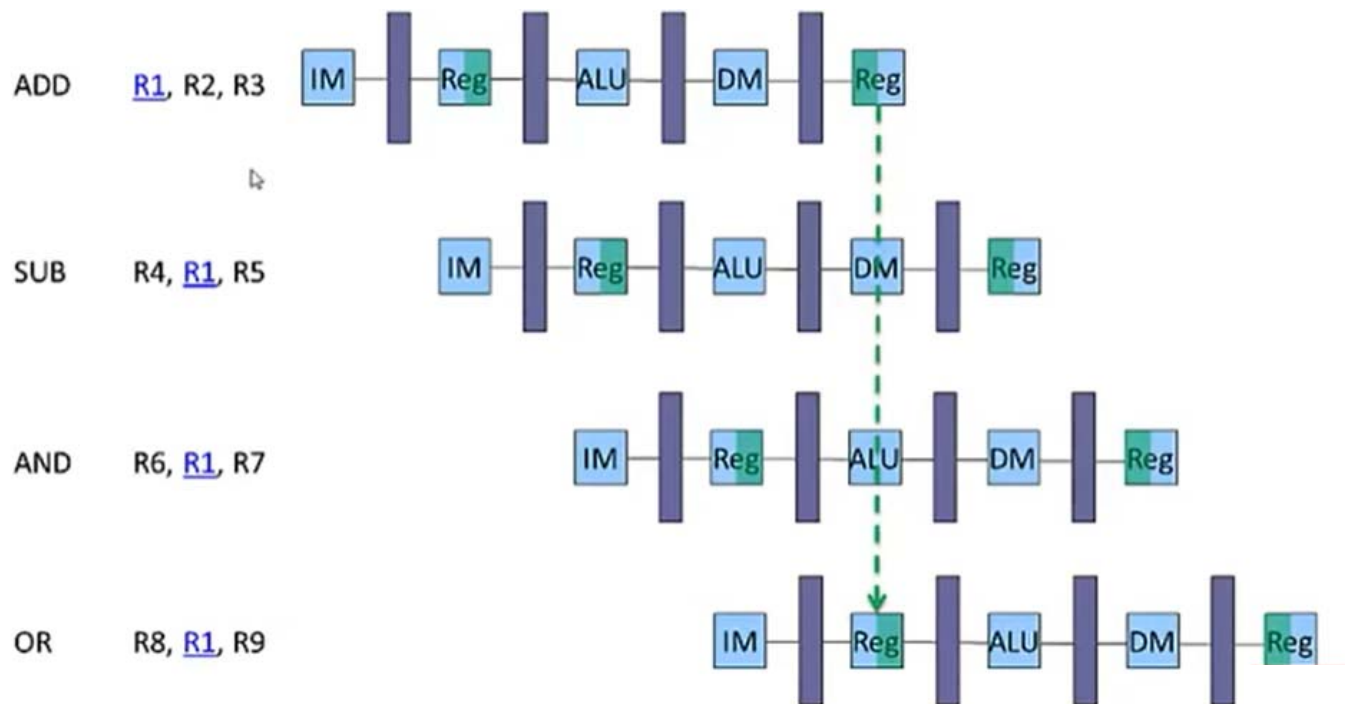
Data Hazards: Another representation of the stall

LW R1, 0(R2)	IF	ID	EX	MEM	WB				
SUB R4, R1, R5		IF	ID	EX	MEM	WB			
AND R6, R1, R7			IF	ID	EX	MEM	WB		
OR R8, R1, R9				IF	ID	EX	MEM	WB	

LW R1, 0(R2)	IF	ID	EX	MEM	WB				
SUB R4, R1, R5		IF	ID	stall	EX	MEM	WB		
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB	
OR R8, R1, R9				stall	IF	ID	EX	MEM	WB

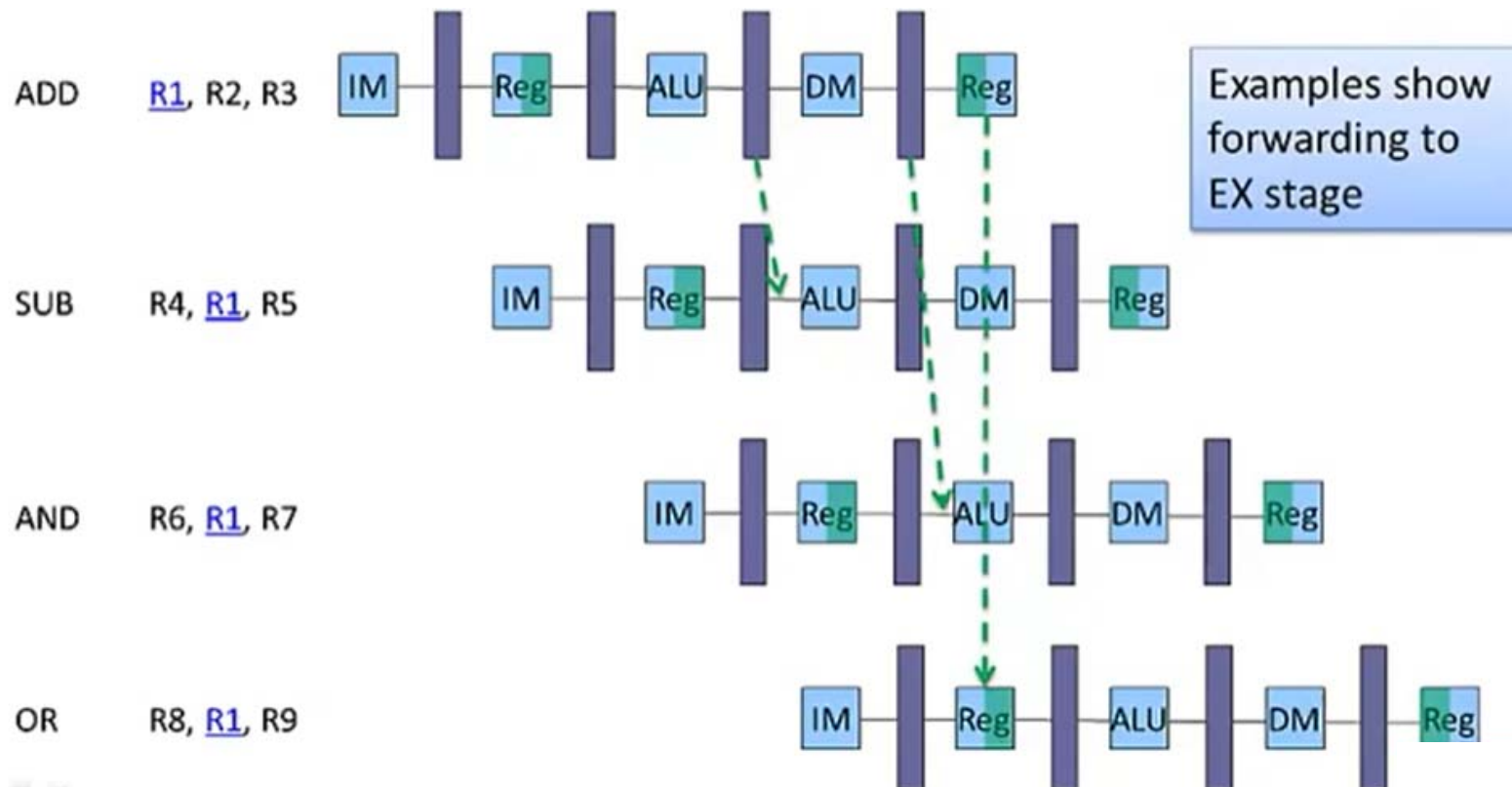
Data Hazard: Can we do better?

One solution can be Read after Write, but it **can't solve all the data hazard in every stage**



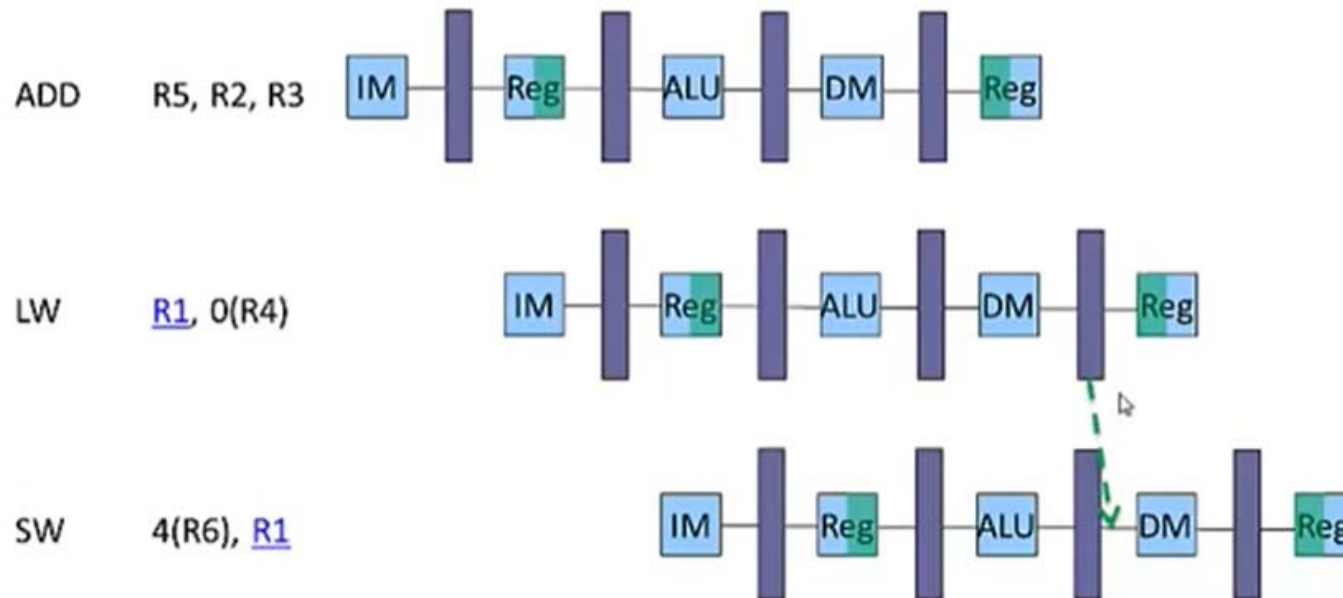
Data Hazard: Can we do better?

- We can minimize stalls by data forwarding that involves all the instructions!



Data Hazard: What if we have lw and sw?

- Data forwarded to Memory stage:



Reordering **Code to Avoid Pipeline Stall** (**Software Solution**)

- Example:

```
lw $t0, 0($t1)
lw $t2, 4($t1)
sw $t2, 0($t1)
sw $t0, 4($t1)
```



Data hazard

- Reordered code:

```
lw $t0, 0($t1)
lw $t2, 4($t1)
sw $t0, 4($t1)
sw $t2, 0($t1)
```



Interchanged

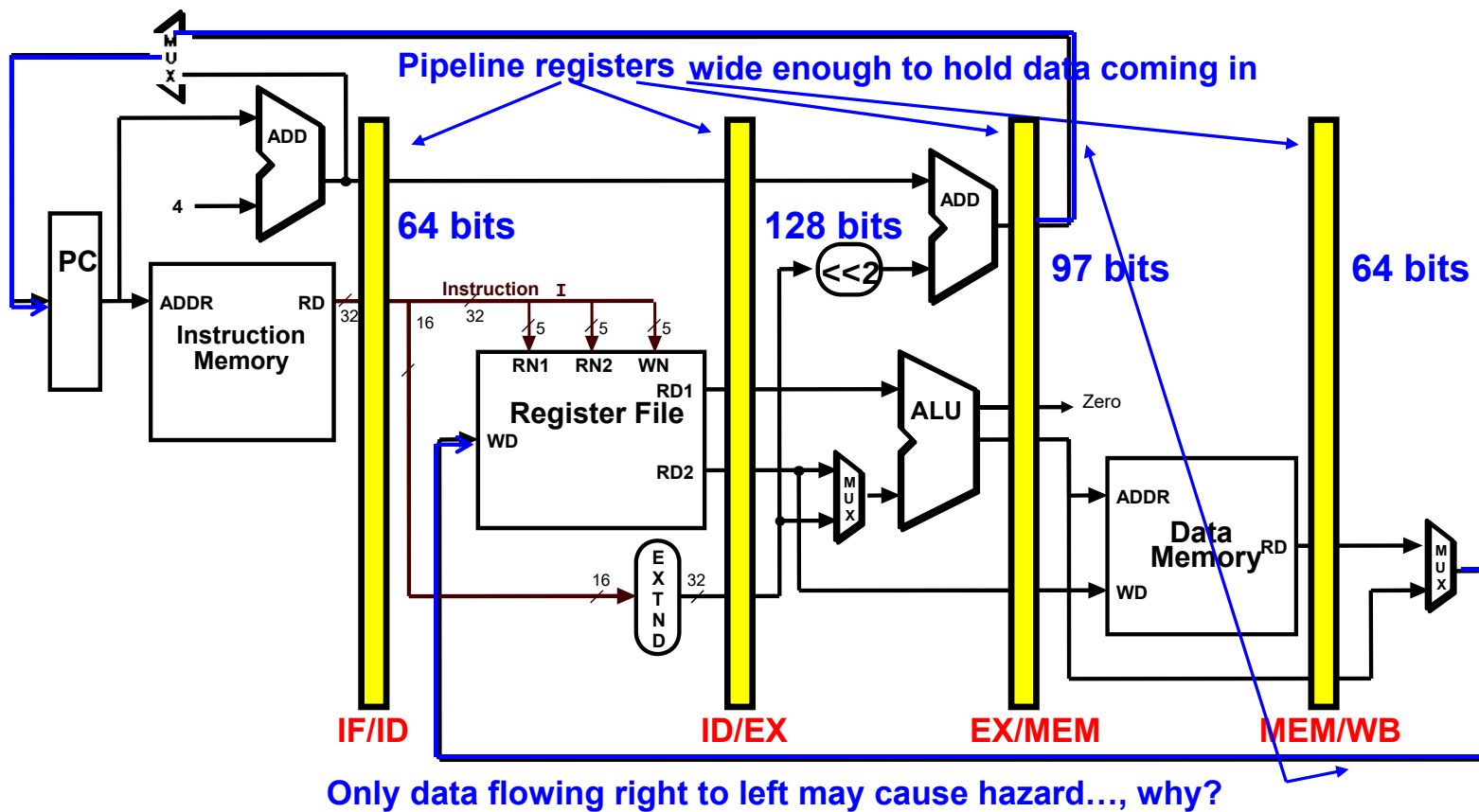
Pipelined Datapath

- We now move to actually building a pipelined datapath
- First recall the 5 steps in instruction execution
 1. Instruction Fetch & PC Increment (IF)
 2. Instruction Decode and Register Read (ID)
 3. Execution or calculate address (EX)
 4. Memory access (MEM)
 5. Write result into register (WB)
- Review: single-cycle processor
 - all 5 steps done in a single clock cycle
 - dedicated hardware required for each step
- *What happens if we break the execution into multiple cycles, but keep the extra hardware?*

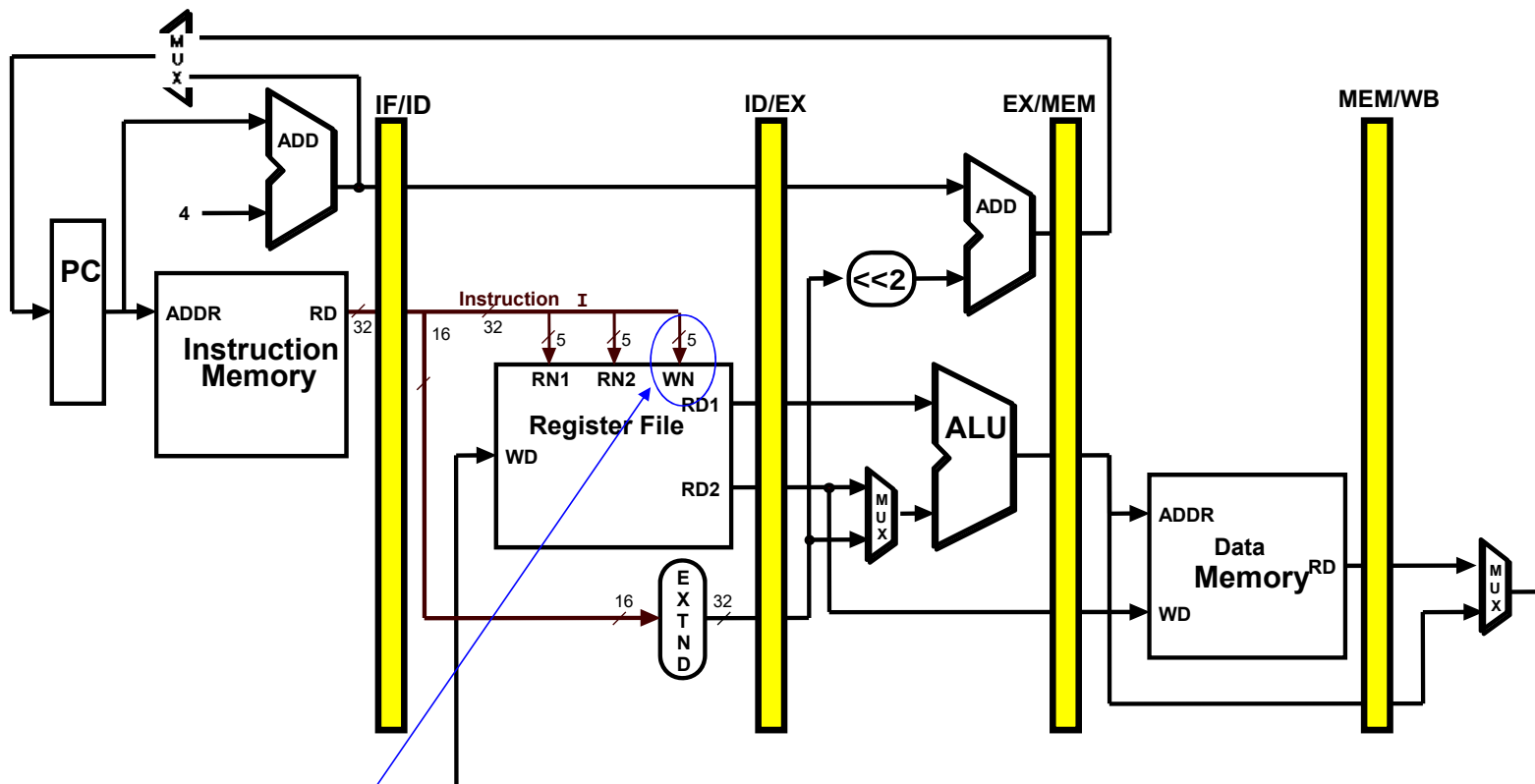
Pipelined Datapath – Key Idea

- *What happens if we break the execution into multiple cycles, but keep the extra hardware?*
 - *Answer: We may be able to start executing a new instruction at each clock cycle - pipelining*
- ...but we shall need extra registers to hold data between cycles – *pipeline registers*

Pipelined Datapath

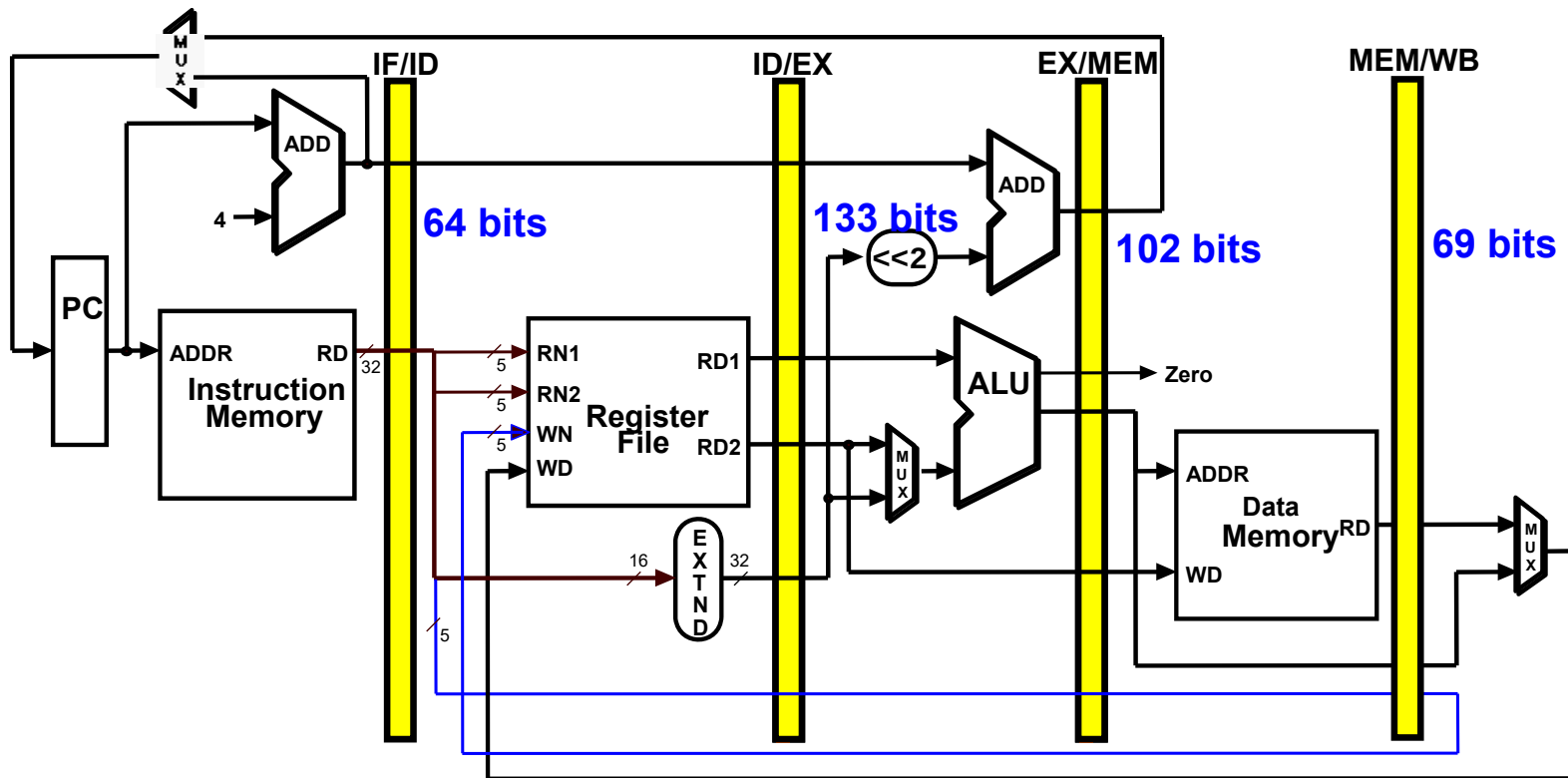


Bug in the Datapath



Write register number comes from another *later* instruction!

Corrected Datapath



Destination register number is also passed through ID/EX, EX/MEM and MEM/WB registers, which are now wider by 5 bits

Superscalar and Dynamic Pipelining

- In the interest of even faster processors, there have been three major directions:
- Super pipelining: It simply means **longer pipelines**.
 - Since the ideal maximum from pipelining is related to the number of pipeline stages, some recent microprocessors speedup have gone to pipelines with **eight or more stages**.
- Superscalar: Usage of **duplicate components** to **replicate the internal components** of the computer so that it can launch multiple instructions in every pipeline stage.
- Dynamic pipeline scheduling / Dynamic pipelining: Dynamic pipelining is normally combined with extra hardware resources so later instructions can proceed in parallel.
 - Divided into three major units:
 - Instruction fetch and issue unit
 - Execute units
 - Commit unit

Dynamic Pipelining Scheduling

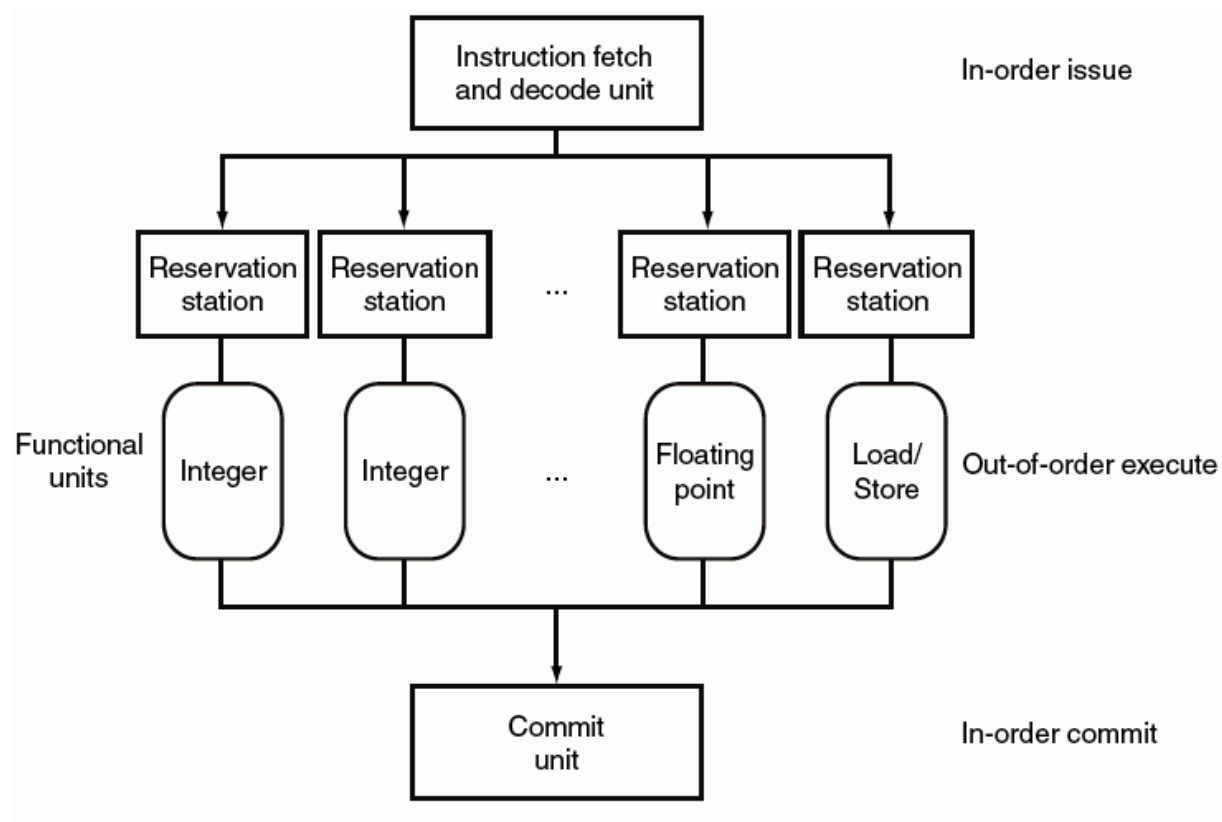


FIGURE 6.49 The three primary units of a dynamically scheduled pipeline.

Superscalar and Dynamic Pipelining

- In-order completion
- Out-of-order completion
- Dynamic pipelining is more complicated than the traditional or static pipelining – why?
 - Dynamic scheduling is normally **combined with branch prediction**, so the commit unit must be able to discard all the results in the execution units that were due to instructions executed after a mispredicted branch.
 - Dynamic scheduling is also typically **combined with superscalar execution**, so each unit may be issuing or committing four to six instructions each clock cycle.